Laborator nr. 3

Interpretoare de comenzi - Bourne Shell (bash). Scripturi

1. Shell-ul ca limbaj de programare

Interpretorul de comenzi Shell permite utilizatorului realizarea unor programe numite *Shell Scripts* facilitând construirea unor secvențe de comenzi ce sunt executate în mod identic cu programele compilate.

Shell-ul are și funcția de limbaj de programare. Recunoaște toate noțiunile de bază specifice oricărui limbaj de programare, cum ar fi: variabile, instrucțiuni de atribuire, instrucțiuni de control (`if, while, for, case`), proceduri/funcții, parametri. Toate aceste facilități permit automatizarea unor acțiuni pe care utilizatorul le desfășoară în mod repetat. Exista posibilitatea de a scriere fișiere care sa contina comenzi, fișiere care pot fi executate la cerere de către interpretorul de comenzi (în terminologia Linux/Unix, un asemenea fișier se numește script).

2. Crearea fișierelor shell script

Scripturile shell sunt fișiere de comenzi Linux/Unix, analog cu fisierele batch *.bat din DOS. Apelul lor se face la fel ca pentru orice comanda Linux/Unix. Recuperarea în script a argumentelor de apel se face cu ajutorul unor variabile speciale (care vor fi parametrii formali în script), pe care le vom vedea mai jos.

În fișierele script se folosește caracterul "`#`" pentru a indica un comentariu, ce este valabil până la sfârșitul acelei linii de text, analog cu forma: //comentariu din limbajul C++.

Lansarea în execuție a unui fișier de comenzi se poate face în mai multe moduri:

a) prin numele lui, la fel ca orice comanda (posibil numai în cazul în care fișierul a fost făcut executabil setându-i dreptul de execuție, i.e. atributul `x`, cu comanda chmod):

```shell
$ nume_fisier_comenzi [parametri]
```

b) prin comanda . a interpretorului de comenzi:

```shell
$ . nume_fisier_comenzi [parametri]
```

c) apeland un anumit shell, de exemplu:

```shell
$ bash nume_fisier_comenzi [parametri]
```

Exista unele diferente semnificative între aceste forme de apel, după cum vom vedea mai târziu (la modul de execuție a proceselor).

Datorită existenței mai multor interpretoare de comenzi, este necesar un mecanism prin care sistemul să poată fi informat asupra interpretorului de comenzi pentru care a fost

scris un anumit fișier de comenzi, în caz contrar acesta riscând să nu poată fi executat. Printr-o convenție respectată de mai multe variante Unix (inclusiv de Linux), prima linie a fișierului de comenzi poate conține numele interpretorului caruia ii este destinat in forma:

```
Shell
#!nume_interpretor_comenzi
```

Exemple:

```
#!/bin/bash
#!/bin/sh
#!/bin/csh
#!/bin/perl
```

Se observa ca trebuie precizata si calea absoluta pentru interpretorul de comenzi cu care se dorește a fi executat acel script. Acest mecanism este folosit și de fișierele care sunt scrise în limbaje mai puternice, dar de același tip (de exemplu `/bin/perl`).

Observație: apelul shell-ului specificat într-un script în felul descris mai sus, pentru a executa acel script, se face doar pentru prima forma de apel:

```
Shell
$ script [argumente]
```

Pentru a doua forma de apel:

```
Shell
$ . script [argumente]
```

scriptul va fi executat de fapt tot de shell-ul în care am dat aceasta comanda.
Pentru a treia forma de apel:

```
Shell
$ shell script [argumente]
```

scriptul va fi executat de shell-ul specificat pe prima poziție a acestui apel.

Pentru ca noul script sa fie executabil, se va folosi una din următoarele comenzi:

```
Shell
$ chmod 755 myscript
```

sau

```shell
$ chmod a+x myscript
```

(vezi comanda `chmod`)

Un shell script este invocat exact ca o comanda Linux obișnuită - prin tastarea numelui ei.

3. Variabile de shell

O alta facilitate comuna tuturor interpretoarelor de comenzi Linux/Unix este utilizarea de variabile. Pentru a crește flexibilitatea sistemului este posibilă definirea, citirea și modificarea de variabile de către utilizator. Trebuie reținut că variabilele sunt numai de tip șir de caractere (excepție făcând parțial interpretorul `csh`).

Instrucțiunea de atribuire are forma:

```shell
$ var=expr
```

unde `var` este un nume (identificator) de variabila, iar `expr` este o expresie care trebuie să se evalueze la un șir de caractere (după ”=” **spațiile nu sunt permise**)

Variabilele sunt păstrate într-o zonă de memorie a procesului shell respectiv, sub forma de perechi nume=valoare. Pentru a vedea ce variabile sunt definite, puteți folosi comanda:

```shell
$ set
```

care va afișa lista acestor perechi.

Comanda:

```shell
$ var=
```

sau

```shell
$ unset var
```

are ca efect nedefinirea variabilei var (adică acea variabila este ștearsă din memorie și orice referire ulterioară la ea va cauza afișarea unui mesaj de eroare.

Referirea la valoarea unei variabile (i.e. atunci când avem nevoie de valoarea variabilei într-o expresie) se face prin numele ei precedat de simbolul `$`, ceea ce cauzează substitutia numelui variabilei prin valoarea ei în expresia în care apare:

`$ echo $var`

Efect: se va afișa valoarea variabilei `var`.

`$ echo var`

Efect: se va afișa șirul de caractere "`var`".

Câteva exemple:

`$ v=a123b`

Efect: operația de atribuire a unei valori variabilei `v`

`$ cat xy$v`

Efect: este echivalent cu: `$ cat xya123b`

`$ v=zz$v`

Efect: variabila `v` este modificată; se observa ca are loc o operație de concatenare.

`$ v=`

Efect: variabila `v` primește valoarea nula (este distrusă).

`$ v=`wc -l fis``

Efect: variabila `v` primește drept valoare ieșirea standard a comenzii specificate între caracterele `...` .

Alte exemple pentru modul de acțiune al caracterelor `...` :

`$ dir-curent=`pwd``

Efect: variabila `dir-curent` va primi ca valoare ieșirea standard a comenzii `pwd`, care este tocmai numele directorului curent (specificat prin cale completa).

```
Shell
$ a=10
$ a=`expr $a + 5`
$ echo $a
```

Efect: se va afișa 15.

Obs: `expr` este o comanda care evalueaza expresii aritmetice și stringuri (vezi detalii în help, adică cu: `$ man expr`).

Pentru substitutia variabilelor prin numele lor se folosesc caracterele `{}` pentru a indica numele variabilei atunci când acesta nu este urmat de spațiu, și anume dacă este de forma:

`${var}sir`

Efectul: se va substitui variabila cu numele `var` si nu cea cu numele `varsir` (cum s-ar fi intamplat daca nu foloseam caracterele `{}`).

```
Shell
$ rad=/home/
$ ls -l ${rad}so
```

Efect: se va lista conținutul directorului `/home/so`

Alte forme de substituție:

`${var:-sir}` = rezultatul expresiei este valoarea variabilei `var`, dacă aceasta este definită, astfel este valoarea `sir`.

`${var:-}` = rezultatul expresiei este valoarea variabilei `var`, dacă aceasta este definită, altfel este afișat un mesaj standard de eroare care spune că acea variabila este

nedefinita.

`${var:=sir}` = rezultatul expresiei este valoarea variabilei `var`, după ce eventual acesteia i se asigneaza valoarea `sir` (asignarea are loc doar în cazul în care `var` era nedefinită).

`${var:?sir}` = rezultatul expresiei este valoarea variabilei `var`, daca aceata este definită, altfel este afișat mesajul `sir` (sau un mesaj standard de eroare, dacă `sir` lipsește).

`${var:+sir}` = dacă variabila `var` este definită (are o valoare), atunci i se asigneaza valoarea `sir`, altfel rămâne în continuare fără valoare (deci asignarea are loc doar în cazul în care `var` era deja definita).

Pe lângă `set`, mai sunt și alte comenzi pentru variabile:

```
$ export var [var2 var3 ...]
```

Efect: are loc "**exportul**" variabilelor în procesele fii ale acelui proces (în mod obișnuit variabilele nu sunt vizibile în procesele fii, ele fiind locale procesului shell respectiv, fiind păstrate în memoria acestuia).

Sau:

```
$ export var=valoare
```

Efect: asignare + export de variabila printr-o singura comanda.

```
$ read var [var2 var3 ...]
```

Efect: citește de la `stdin` valori și le atribuie variabilelor specificate.

```
$ readonly var [var2 var3 ...]
```

Efect: acele variabile sunt declarate ca fiind read-only (i.e. nu mai pot fi modificate după aceasta comanda, rămân cu valoarea pe care o aveau când s-a executat aceasta comanda).

În terminologia Linux/Unix, se folosesc termenii de **variabila de shell** si **variabila de mediu**, cu semnificații diferite:

variabila de shell este o variabila accesibilă doar procesului curent;

variabila de mediu este o variabila accesibilă tuturor proceselor fii ale acelui proces shell (atunci când este o **variabila exportată**).

Exista o serie de variabile ce sunt modificate dinamic de către shell, pentru a le păstra semnificația pe care o au, și care pot fi folosite în scripturi, în conformitate cu semnificația lor:

1) `$0` = numele procesului curent (numele scriptului în care este referita)

2) `$1`,...,`$9` = parametrii cu care a fost apelat procesul curent (i.e. parametrii din linia de apel în cazul unui script);

Observație: un fișier de comenzi poate primi, la lansarea in executie, o serie de **parametri din linia de comanda**. Aceștia sunt referiti în corpul fișierului prin denumirile `$1,$2,...,$9`.

Interpretorul de comenzi pune la dispoziția utilizatorului comanda interna `shift`, prin care se poate realiza o deplasare a valorilor parametrilor din linia de comanda: vechea valoare a lui `$2` va fi referita prin `$1`, vechea valoare a lui `$3` va fi referita prin `$2` s.a.m.d., iar vechea valoare a lui `$1` se pierde.

Comanda `shift` este utilă pentru cazurile când avem mai mult de 9 parametri in linia de comanda, pentru a avea acces la toți aceștia.

3) `$*` = lista parametrilor din linia de comanda (fără argumentul `$0`);

4) `$#` = numărul acestor parametri;

5) `$$` = pid-ul procesului curent (i.e. pid-ul shell-ului ce executa acel script);

Observație: variabila `$$` poate fi folosită pentru a crea fișiere temporare cu nume unic, de exemplu: fișierul cu numele `/tmp/err$$`.

6) `$?` = codul returnat de ultima comanda shell executată;

Observație: la fel ca în DOS, și în Unix fiecare comanda returnează un cod de retur la terminarea ei, care este 0, în caz de succes, sau este o valoare nenulă (codul erorii), în cazul unei erori.

7) `$!` = pid-ul ultimului proces executat în background;

8) `$-` = opțiunile cu care a fost lansat procesul shell respectiv. Aceste opțiuni pot fi:

a) `-x` = afiseaza linia din script executata (istoric al ultimelor comenzi executate);

b) `-v` = modul verbose de execuție (listeaza toate comenzile executate);

Aceste opțiuni se pot folosi cu comanda `set`:

a) `set -v ; script` -> util pentru depanare: setez întâi opțiunea -v si apoi execut scriptul specificat in modul verbose;

b) `set -x` -> util pentru a vedea istoricul (ultimele comenzi executate);

c) `set -u` -> verifica dacă variabilele au fost inițializate;

d) `set -n` -> anulează toate comenzile ce urmează după ea până la sfârșitul scriptului (aceasta comanda se folosește în interiorul scriptului);

e) `set -i` -> shell-ul devine interactiv (util pentru depanare).

Alt exemplu pentru modul de acțiune al caracterelor `` `...` `` : recuperarea rezultatului unei comenzi pe câmpuri în variabilele `$1,$2,...,$9` :

```shell
$ set `date` ; echo $6 $2 $3 $4
```

Efect: prima comanda determina initializarea variabilelor `$1,$2,...,$9` cu cimpurile rezultatului comenzii (observație: câmpurile sunt subcuvintele din ieșirea standard a comenzii ce sunt separate prin spații). A doua comanda va afișa anul (câmpul 6), luna (câmpul 2), ziua (câmpul 3) și ora (câmpul 4) curente.

```shell
$ set a b c
```

Efect: determina crearea si initializarea variabilelor `$1, $2, $3` respectiv cu valorile `a, b, c`.

Variabilele speciale de tip `$` sunt:

`$?name` = 0 dacă numele nu este stabilit
= 1 dacă numele este stabilit

`$name[1-3]` = primele 3 elemente ale numelui de tip array

`$name[2-]` = toate elementele de la al doilea până la ultimul din numele respectiv

`$name[*]` = toate elementele numelui

`$<` = citește linia 1 a intrării de la terminal

Variabile de tip argv (pentru shell-ul csh):

Variabilele de tip array argv primesc argumentele destinate scriptului. Acestea

permit un alt procedeu de referire a argumentelor similar cu `$1`, `$2`, etc.

`$#argv` = numărul de elemente din argv
`$argv[1]` = primul element al variabilei de tip array argv (`$1`)
`$argv[$#argv]` = ultima valoare a variabilei de tip array argv
`$argv[1-3]` = primele trei argumente

Alte exemple:

a) `$ who | grep an2&so`
Efect: se va interpreta caracterul **&** drept execuție în background;

b) `$ who | grep an2\&so` sau `$ who | grep "an2&so"`
Efect: se va interpreta caracterul `&` prin el însuși și nu ca fiind execuție în background;

c) `$ who | grep "an2$PATH"`
Efect: se va substitui variabila `$PATH` cu valoarea sa;

d) `$ who | grep 'an2$PATH'`
Efect: nu se va substitui variabila `$PATH` cu valoarea sa.

De asemenea, exista o serie de variabile de mediu **predefinite** (au anumite semnificații fixate, aceleași pentru toata lumea), și anume:

1) `$HOME` = directorul de login al acelui utilizator;
2) `$USER` = username (numele de login) al acelui utilizator;
3) `$LOGNAME` = la fel ca `$USER`;
4) `$SHELL` = shell-ul implicit al acelui utilizator;
5) `$MAIL` = numele complet al fișierului de posta electronica al acelui utilizator (este utilizat de shell pentru a ne anunta daca a fost primit un nou mesaj, necitit încă, adică acel binecunoscut mesaj "`You have new mail in $MAIL`";
6) `$PS1` = șirul de caractere al prompterului principal asociat shell-ului;
7) `$PS2` = șirul de caractere al prompterului secundar asociat shell-ului (prompterul secundar este folosit pentru liniile de continuare ale unei comenzi scrise pe mai multe linii);
8) `$TERM` = specifica tipul de terminal utilizat (vt100, vt102 s.a.);
9) `$PATH` = o lista de directoare în care shell-ul cauta fisierul corespunzator unei comenzi tastate (specificata doar prin nume, nu si prin cale, absoluta sau relativa);
10) `$CDPATH` = o lista de directoare în care shell-ul cauta directorul dat ca parametru comenzii cd, în cazul când acesta a fost specificat doar prin nume, nu si prin cale (absoluta sau relativa); similar ca `PATH` pentru fisiere;
11) `$IFS` = specifica mulțimea caracterelor ce sunt interpretate ca spațiu.

Mai sunt si alte variabile de mediu (le puteți vedea pe toate utilizând comanda `set` fără parametri). Aceste variabile sunt inițializate de către shell la deschiderea unei sesiuni de lucru, cu valorile specificate în fișierele de inițializare ale sistemului (de fapt sunt exportate, după cum puteți constata citind aceste fișiere).

În continuare vom prezenta aceste fișiere de inițializare (de configurare) a sistemului.

4. Fișierele de configurare

Fiecare utilizator își poate scrie un script care sa fie executat la fiecare început de sesiune de lucru (analogul fisierului `autoexec.bat` din DOS), script numit `$HOME/.profile sau $HOME/.bash_profile` în cazul când se utilizează `bash` ca shell implicit (pentru alte shell-uri este denumit altfel). În plus poate avea un script care sa fie rulat atunci când se deconectează de la sistem (adică la `logout`); acest script se

numeste `$HOME/.bash_logout` in cazul shell-ului `bash`. După cum observați, toate aceste fișiere se găsesc în directorul `HOME` al acelui utilizator.

Pentru a testa modul de execuție a fișierelor de inițializare, folosiți comenzi de afișare pe ecran a unor mesaje. De exemplu, adăugați în fiecare fișier la sfârșit câte o linie de forma:

```
echo "Executing file .profile"
```

respectiv

```
echo "Executing file .bash_profile"
```

(Dacă vreunul dintre fișiere nu exista, acesta trebuie creat.)

Apoi deschideți o nouă sesiune de lucru și urmăriți mesajele afișate pe ecran.

Mai exista doua fișiere de inițializare valabile pentru toți utilizatorii, și anume fișierele `/etc/profile` și `/etc/environment`. La deschiderea unei noi sesiuni, întâi sunt executate scripturile de sistem (din `/etc/`) și abia apoi cele particulare utilizatorului respectiv (din `$HOME/`).

Mai exista niște fișiere de configurare, și anume `/etc/bashrc` și `$HOME/.bashrc` (în cazul shell-ului bash), care sunt executate ori de câte ori este lansat un proces shell interactiv (Exemplu: atunci când de sub browserul `lynx` apelați interpretorul de comenzi printr-o anumită comandă [mai exact, cu tasta: "!"], sau dintr-un editor de texte etc.)

Mai exista un fișier, numit `$HOME/.bash_history`, care pastreaza ultimele N comenzi tastate (exista o variabila de mediu care specifica aceasta dimensiune N). Ele pot fi vizualizate cu comanda:

```shell
$ history
```

5. Structuri de control pentru scripturi

Fiecare interpretor de comenzi furnizeaza o serie de structuri de control de nivel înalt, ceea ce confera fișierelor de comenzi o putere mult mai mare decât este posibil în DOS. Cele mai utilizate structuri de control ale interpretorului bash sunt: `for, while, until, if, case`. (**Observatie**: acestea sunt comenzi interne ale shell-ului bash.)

1) Bucla iterativa `for` are sintaxa:

```
for <var> [ in <text> ]
do
  <lista_comenzi>
done
```

Semantica: `<text>` descrie o listă de valori pe care le ia succesiv variabila `<var>`; pentru fiecare asemenea valoare a lui `<var>`, se executa comenzile din `<lista_comenzi>`.

Observații:

a) Se poate folosi în corpul buclei `for` valoarea variabilei `<var>`;

b) Toate instrucțiunile, exceptând `do` și `done`, trebuie sa fie urmate de caracterul "`;`" sau caracterul newline;

c) Dacă lipsește partea opțională "`în <text>`", atunci ca valori pentru `<var>` se folosesc argumentele din `$*` (i.e. parametrii din linia de comanda).

Exemplu:

```shell
for i in `ls -t`
do
    echo $i
done
```

sau:

```shell
for i in `ls -t` ; do echo $i ; done
```

Efect: același cu comanda `ls -t` (se afișează continutul directorului curent, sortat dupa data).

2) Bucla repetitiva `while` are sintaxa:

```
while <lista_comenzi_1>
 do
   <lista_comenzi_2>
 done
```

Semantica: Se executa comenzile din `<lista_comenzi_1>` și dacă codul de retur al ultimei comenzi din ea este 0 (i.e. terminare cu succes), atunci se executa comenzile din `<lista_comenzi_2>` și se reia bucla. Altfel, se termina execuția buclei while.

Observații:

a) Deci bucla `while` se executa iterativ atât timp cât codul returnat de `<lista_comenzi_1>` este 0 (succes);

b) Adeseori `<lista_comenzi_1>` poate fi comanda:

```
test <argumente>
```

sau, echivalent:

```
[ <argumente> ]
```

care este o comanda ce exprima testarea unei condiții, după cum vom vedea mai jos.

Exemplu:

```shell
while true
do
  date;
  sleep 60;
done
```

Efect: se afișează încontinuu pe ecran, din minut în minut, data și ora curentă.

3) Bucla repetitiva `until` are sintaxa:

```
until <lista_comenzi_1>
 do
```

```
    <lista_comenzi_2>
  done
```

Semantica: Se executa comenzile din `<lista_comenzi_1>` și dacă codul de retur al ultimei comenzi din ea este diferit de 0 (i.e. terminare cu eroare), atunci se executa comenzile din `<lista_comenzi_2>` și se reia bucla. Altfel, se termina execuția buclei `until`.

Observații:

a) Deci bucla until se executa iterativ atât timp cât codul returnat de `<lista_comenzi_1>` este diferit de 0 (eroare), sau cu alte cuvinte bucla until se executa iterativ până când codul returnat de `<lista_comenzi_1>` este 0 (succes);

b) Adeseori `<lista_comenzi_1>` poate fi comanda de testare a unei condiții.

4) Structura alternativa `if` are sintaxa:

```
 if <lista_comenzi_1>
  then
    <lista_comenzi_2>
[ else
    <lista_comenzi_3> ]
  fi
```

Semantica: Se executa comenzile din `<lista_comenzi_1>` și dacă codul de retur al ultimei comenzi din ea este 0 (i.e. terminare cu succes), atunci se executa comenzile din `<lista_comenzi_2>`, iar altfel (i.e. codul de retur este diferit de 0) se executa comenzile din `<lista_comenzi_3>`.

Observații:

a) Adeseori `<lista_comenzi_1>` poate fi comanda de testare a unei condiții;

b) Ramura else este opțională;

c) Structura `if` are și o forma sintactica imbricata:

```
if <lista_comenzi_1>
then
  <lista_comenzi_2>
elif <lista_comenzi_3>
then
  <lista_comenzi_4>
elif <lista_comenzi_5>
.....
else
  <lista_comenzi_N>
fi
```

5) Structura alternativa `case` are sintaxa:

```
case <expr> in
  <sir_valori_1>   ) <lista_comenzi_1>   ;;
  <sir_valori_2>   ) <lista_comenzi_2>   ;;
  .....
  <sir_valori_N-1> ) <lista_comenzi_N-1> ;;
  <sir_valori_N>   ) <lista_comenzi_N>
esac
```

Semantica: Dacă valoarea `<expr>` se găsește în lista de valori `<sir_valori_1>`, atunci se executa `<lista_comenzi_1>` și apoi execuția lui case se termina. Altfel: dacă valoarea `<expr>` se găsește în lista de valori `<sir_valori_2>`, atunci se executa `<lista_comenzi_2>` și apoi execuția lui case se termina. Altfel: ... s.a.m.d.

Observații:

a) Deci se intra în prima listă de valori cu care se potrivește, fără a se verifica unicitatea (dacă mai sunt și alte liste de valori cu care se potrivește);
b) Ultima linie dinainte de esac nu este obligatoriu sa se termine cu caracterele "`;;`" ;
c) Lista de valori `<sir_valori_X>` poate fi de forma:

`<val_1> | <val_2> | ... | <val_M>` (deci o enumerare de valori),

sau poate fi o expresie regulată, ca de exemplu:

```
case $opt in
  -[ax-z] )  <comenzi> ;;
  ...
esac
```

care este echivalent cu:

```
case $opt in
  -a|-x|-y|-z )  <comenzi> ;;
  ...
esac
```

Exemplu:

```shell
case $var1 in
  *.c ) var2=`basename $var1 .c` ; gcc $var1 -o$var2 ;;
  ...
esac
  echo Source $var1 was compiled into executable $var2.
```

Efect: dacă variabila var1 are ca valoare "`fisier.c`" (i.e. numele unui fisier sursa), atunci variabila `var2` va avea ca valoare "`fisier`" și apoi fisierul sursa `$var1` este compilat obținându-se un executabil cu numele `$var2`.

Observație: comanda basename `<arg1> <arg2>` returnează în ieșirea standard valoarea argumentului `<arg1>` după ce se inlatura din el sufixul `<arg2>`.

IMPORTANT: aceste structuri de control fiind comenzi interne, puteți folosi comanda de **`help`** pentru comenzi interne:

```shell
$  help <structura>
```

pentru a afla mai multe detalii despre fiecare structura în parte.

6. Alte comenzi

Comanda de testare a unei condiții este comanda (predicatul) `test`, având forma:

```
test <conditie>
```

sau:

```
[ <conditie> ]
```

unde condiția `<conditie>` poate fi:

a) o comparatie intre doua șiruri de caractere (utilizând simbolurile "==" și "`!=`", cunoscute din limbajul C):

```
$  test <expr_1> == <expr_2>
```

Efect: returnează `true` (codul de retur 0) dacă cele două expresii au aceeași

valoare, altfel returnează `false` (cod de retur nenul);

`$ test <expr_1> != <expr_2>`

Efect: returnează `true` (codul de retur 0) dacă cele două expresii au valori diferite, altfel returnează false (cod de retur nenul);

b) condiții relaționale:

`$ test <val_1> -<rel> <val_2>`

Efect: returnează `true` (codul de retur 0) dacă valoarea `<val_1>` este în relația `<rel>` cu valoarea `<val_2>`, unde `<rel>` este unul dintre operatorii relationali următori:

- `eq` = equal (egal: =)
- `gt` = greater-than (mai mare decât: >)
- `ge` = greater-equal (mai mare sau egal cu: >=)
- `lt` = less-than (mai mic decât: <)
- `le` = less-equal (mai mic sau egal cu: <=)

c) una din următoarele condiții:

`$ test -e <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă exista un fișier de orice fel (obișnuit, director, special, etc.) având numele <nume_fisier>;

`$ test -d <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă `<nume_fisier>` este un director;

`$ test -f <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă `<nume_fisier>` este un fișier obișnuit;

`$ test -p <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă `<nume_fisier>` este un fișier de tip pipe;

`$ test -b <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă `<nume_fisier>` este un fișier de tip dispozitiv în mod bloc;

`$ test -c <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă `<nume_fisier>` este un fișier de tip dispozitiv în mod caracter;

`$ test -s <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă fișierul `<nume_fisier>` este nevid (i.e. are lungimea mai mare decât 0);

`$ test -r <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă fișierul `<nume_fisier>` poate fi citit (i.e. are setat atributul r);

`$ test -w <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă fișierul `<nume_fisier>` poate fi modificat (i.e. are setat atributul w);

`$ test -x <nume_fisier>`

Efect: returnează true (codul de retur 0) dacă fișierul `<nume_fisier>` poate fi lansat in executie (i.e. are setat atributul x);

d) o expresie logica: negație, conjuncție, disjuncție:

`$ test !<conditie_1>`

Efect: NOT : negația condiției `<conditie_1>`;

`$ test <conditie_1> -a <conditie_2>`

Efect: AND : conjunctia condițiilor `<conditie_1>` și `<conditie_2>`;

`$ test <conditie_1> -o <conditie_2>`

Efect: OR : disjuncția condițiilor `<conditie_1>` si `<conditie_2>`; unde `<conditie_1>` și `<conditie_2>` sunt condiții de una din formele a), b), c) sau d).

Exemplu:

```shell
#!/bin/bash
for i in *
do
  if test -f $i
  then
    echo $i
  fi
done
```

Efect: acest script listeaza fișierele obișnuite din directorul curent.

Observație: caracterul * joacă un rol special: în evaluare caracterul * se înlocuiește cu numele oricărui fișier din directorul curent (exceptandu-le pe cele al căror nume începe cu caracterul "."). Pentru ca * sa nu se evalueze în acest fel, ci sa se evalueze prin el însuși, trebuie sa se utilizeze apostroful: '*' ; am văzut și mai sus un exemplu (si anume: `who | grep 'an2$PATH'`) în care apostroful determina evaluarea caracterelor speciale prin ele însele.

Alte comenzi (instrucțiuni) ce pot apare în scripturi:

1) comanda `break`, cu sintaxa:

`break [n]`

unde n este 1 în caz ca lipsește.

Efect: se iese din n bucle `do-done` imbricate, execuția continuând cu următoarea instrucțiune de după done.

2) comanda `continue`, cu sintaxa:

`continue [n]`

unde n este 1 în caz ca lipsește.

Efect: pentru `n=1` se reâncepe bucla curentă do-done (de la pasul de reinitializare), respectiv pentru `n>1` efectul este ca și cum se executa de n ori comanda continue 1.

3) comanda `exit`, cu sintaxa:

`exit cod`

Efect: se termina (se oprește) execuția scriptului în care apare și se întoarce drept cod de retur valoarea specificata.

4) comanda `exec`, cu sintaxa:

`exec <lista_comenzi>`

Efect: se executa comenzile specificate fără a se creea o nouă instanță de shell (astfel shell-ul ce executa aceasta comanda se va reacoperi cu procesul asociat comenzii, deci nu este reentrant).

5) comanda `wait`, cu sintaxa:

`wait pid`

Efect: se întrerupe execuția scriptului curent, asteptându-se terminarea procesului cu pid-ul specificat.

6) comanda `eval`, cu sintaxa:

`eval parametri`

Efect: se evalueaza parametrii specificati.

7) comanda `export`, cu sintaxa:

`export variabile`

Efect: se exportă variabilele specificate.

8) comanda `trap`, cu sintaxa:

`trap <comanda> <eveniment>`

Efect: cand se va produce evenimentul specificat (se va primi semnalul respectiv) se va executa comanda specificata.

Evenimente (semnale):

- semnalul 1 = hang-up signal;
- semnalul 2 = interrupt signal (semnal generat prin apasarea tastelor ^C);
- semnalul 3 = quit signal (semnal generat prin apasarea tastelor ^D);
- semnalul 9 = kill signal (semnal ce "omoara" procesul);
- semnalul 15 = semnal de terminare normala a unui proces;
- etc.

Mai multe detalii vom vedea în lecția despre semnale Unix.

Exemplu:

```shell
$  trap 'rm /tmp/ps$$ ; exit' 2
```

Efect: când se va primi semnalul 2, se va șterge fișierul temporar `/tmp/ps$$` și apoi se va termina execuția scriptului respectiv.

7. Execuție condițională

Exista doua posibilitati de a condiționa execuția unei comenzi de rezultatul execuției unei alte comenzi, și anume:

a) prima forma este:

`<comanda_1> && <comanda_2>`

Efect: intâi se executa `comanda 1` și apoi `comanda 2` se va executa doar dacă execuția `comenzii 1` întoarce codul de retur 0 (succes).

b) a doua forma este:

`<comanda_1> || <comanda_2>`

Efect: intâi se executa `comanda 1` și apoi `comanda 2` se va executa doar dacă execuția `comenzii 1` întoarce un cod de retur nenul (eroare).

8. Probleme propuse pentru rezolvare:

8.1. Scrieți un script care afișează parametrii primiți în linia de comanda în ordine inversa indiferent de tipul sau numărul acestora.

Restricții:

- argumentele vor fi preluate și salvate într-un tablou (“Advanced Bash−Scripting Guide”, Capitolul 27).
- afișarea argumentelor se face prin parcurgerea inversă a tabloului.
- scriptul va afișa pe ecran un mesaj corespunzător dacă nu are niciun argument primit în linia de comandă.

8.2. Sa se scrie un script care implementează operațiile matematice de baza (+, -, *, /, %). Scriptul va primi ca parametri 2 operanzi. Scriptul afișează un meniu de unde se poate selecta una din operațiile disponibile. Daca utilizatorul selectează alta optiune, scriptul isi va incheia executia afisand un mesaj de terminare. Operațiile vor fi

implementate prin funcții scrise într-un fișier separat. Scriptul va testa existența fișierului de funcții iar dacă acesta exista îl va încărca, altfel va afișa un mesaj de eroare. Pentru meniu trebuie scrisă o funcție separată.

Restricții:

- funcția pentru afișarea meniului nu se scrie în fișierul ce conține funcțiile pentru operații.
- pentru implementarea meniului se va folosi o structura de control de tip `case`.
- maxim 2 fișiere (scriptul principal și fișierul de funcții matematice)

8.3. Folosind utilitarul **bc** ca în exemplele de mai jos, scrieți doua functii de conversie zecimal-hexa și hexa-zecimal. Modificați scriptul realizat pentru problema anterioara, astfel incat daca scriptul primește un singur argument in linia de comanda, singurele operații disponibile afișate sa fie cele de conversie.

Exemplu de conversie:

```shell
$ echo "ibase=16; FF"|bc;
255
$ echo "obase=16; 255"|bc;
FF
```

Exemplu de apelare corecta a scriptului:

```shell
$ ./script 1 2
$ ./script FF
$ ./script 255
```

9. Bibliografie:

- Mendel Cooper, *Advanced Bash-Scripting Guide*
  http://tldp.org/LDP/abs/abs-guide.pdf
  http://tldp.org/LDP/abs/html/
- Machtelt Garrels, *Bash Guide for Beginners*, 2008
  http://tldp.org/LDP/Bash-Beginners-Guide/Bash-Beginners-Guide.pdf
  http://tldp.org/LDP/Bash-Beginners-Guide/html/
- Mihai Budiu - Elemente de programare în Shell-ul Unix
  http://www.cs.cmu.edu/~mihaib/articles/shell-2/shell-2-html.html
- Gareth Anderson, *GNU/Linux Command-Line Tools Summary*, 2006
  http://tldp.org/LDP/GNU-Linux-Tools-Summary/GNU-Linux-Tools-Summary.pdf
  http://www.tldp.org/LDP/GNU-Linux-Tools-Summary/html/
- http://en.wikipedia.org/wiki/Environment_variable
- BASH Frequently-Asked Questions
  http://www.faqs.org/faqs/unix-faq/shell/bash/
- William E. Shotts Jr., Writing shell scripts
  http://linuxcommand.org/lc3_writing_shell_scripts.php